

Name - Harshit Vats
Superset Id - **7748373**
VIT Bhopal

Data-structures-Algorithms

Exercise 2:E-commerce Platform Search Function

Code:

```
class Product {  
  
    int productId;  
  
    String productName;  
  
    String category;  
  
    Product(int productId, String productName, String category) {  
  
        this.productId = productId;  
  
        this.productName = productName;  
  
        this.category = category;  
  
    }  
  
    @Override  
  
    public String toString() {  
  
        return "Product{ID=" + productId + ", Name='" + productName +  
"', Category='" + category + "'}";  
  
    }  
}  
  
public class EcommerceSearch {
```

```
static Product linearSearch(Product[] products, String productName)
{
    for (int i = 0; i < products.length; i++) {
        if (products[i].productName.equalsIgnoreCase(productName)) {
            return products[i];
        }
    }

    return null;
}

static Product binarySearch(Product[] sortedProducts, String
productName) {
    int left = 0;
    int right = sortedProducts.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        int cmp =
sortedProducts[mid].productName.compareToIgnoreCase(productName);

        if (cmp == 0) {
            return sortedProducts[mid];
        } else if (cmp < 0) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return null;
}
```

```

    }

    static Product[] sortByProductName(Product[] products) {
        Product[] sorted = products.clone();

        for (int i = 0; i < sorted.length - 1; i++) {
            for (int j = 0; j < sorted.length - i - 1; j++) {
                if (sorted[j].productName.compareToIgnoreCase(sorted[j +
1].productName) > 0) {
                    Product temp = sorted[j];
                    sorted[j] = sorted[j + 1];
                    sorted[j + 1] = temp;
                }
            }
        }

        return sorted;
    }

    public static void main(String[] args) {
        System.out.println("=== E-commerce Platform Search Function
===\n");

        Product[] products = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Smartphone", "Electronics"),
            new Product(103, "Headphones", "Accessories"),
            new Product(104, "Keyboard", "Accessories"),
            new Product(105, "Monitor", "Electronics")
        };

        System.out.println("--- All Products ---");

        for (Product p : products) {
            System.out.println(p);
        }
    }
}

```

```

    }

    System.out.println("\n--- Linear Search ---");

    String searchName = "Headphones";

    long startTime = System.nanoTime();

    Product result1 = linearSearch(products, searchName);

    long endTime = System.nanoTime();

    System.out.println("Searching for: " + searchName);

    System.out.println("Result: " + result1);

    System.out.println("Time taken: " + (endTime - startTime) + "
ns");

    // Binary Search (requires sorted array)

    System.out.println("\n--- Binary Search ---");

    Product[] sortedProducts = sortByProductName(products);

    System.out.println("Sorted products:");

    for (Product p : sortedProducts) {

        System.out.println("  " + p);

    }

    startTime = System.nanoTime();

    Product result2 = binarySearch(sortedProducts, searchName);

    endTime = System.nanoTime();

    System.out.println("\nSearching for: " + searchName);

    System.out.println("Result: " + result2);

    System.out.println("Time taken: " + (endTime - startTime) + "
ns");

    System.out.println("\n--- Time Complexity Comparison ---");

    System.out.println("Linear Search: O(n) - No sorting needed,
simple implementation");

```

```

        System.out.println("Binary Search: O(log n) - Requires sorted
data, faster for large datasets");

        System.out.println("\nRecommendation: Use Binary Search for
e-commerce platforms");

        System.out.println("because product catalogs are typically large
and can be pre-sorted.");
    }
}

```

Output:

```

genc-dn5.0/week1/dsa on 1 main [?] via v23.0.2
● > javac EcommerceSearch.java && java EcommerceSearch
=== E-commerce Platform Search Function ===

--- All Products ---
Product{ID=101, Name='Laptop', Category='Electronics'}
Product{ID=102, Name='Smartphone', Category='Electronics'}
Product{ID=103, Name='Headphones', Category='Accessories'}
Product{ID=104, Name='Keyboard', Category='Accessories'}
Product{ID=105, Name='Monitor', Category='Electronics'}

--- Linear Search ---
Searching for: Headphones
Result: Product{ID=103, Name='Headphones', Category='Accessories'}
Time taken: 10541 ns

--- Binary Search ---
Sorted products:
Product{ID=103, Name='Headphones', Category='Accessories'}
Product{ID=104, Name='Keyboard', Category='Accessories'}
Product{ID=101, Name='Laptop', Category='Electronics'}
Product{ID=105, Name='Monitor', Category='Electronics'}
Product{ID=102, Name='Smartphone', Category='Electronics'}

Searching for: Headphones
Result: Product{ID=103, Name='Headphones', Category='Accessories'}
Time taken: 5625 ns

--- Time Complexity Comparison ---
Linear Search: O(n) - No sorting needed, simple implementation
Binary Search: O(log n) - Requires sorted data, faster for large datasets

Recommendation: Use Binary Search for e-commerce platforms
because product catalogs are typically large and can be pre-sorted.

```

Exercise 7: Financial Forecasting

code :

```
public class FinancialForecasting {  
    static double futureValueRecursive(double presentValue, double  
growthRate, int years) {  
        if (years == 0) {  
            return presentValue;  
        }  
        return futureValueRecursive(presentValue * (1 + growthRate),  
growthRate, years - 1);  
    }  
    static double futureValueIterative(double presentValue, double  
growthRate, int years) {  
        double futureValue = presentValue;  
        for (int i = 0; i < years; i++) {  
            futureValue *= (1 + growthRate);  
        }  
        return futureValue;  
    }  
    static double[] memo = new double[100];  
    static double futureValueMemoized(double presentValue, double  
growthRate, int years) {  
        if (years == 0) {  
            return presentValue;  
        }  
        if (memo[years] != 0) {  
            return memo[years];  
        }  
    }  
}
```

```

        memo[years] = futureValueMemoized(presentValue, growthRate,
years - 1) * (1 + growthRate);

        return memo[years];
    }

    public static void main(String[] args) {

        System.out.println("=== Financial Forecasting Tool ===\n");

        double presentValue = 10000.0;

        double growthRate = 0.08; // 8% annual growth

        int years = 10;

        System.out.println("--- Parameters ---");

        System.out.println("Present Value: $" + presentValue);

        System.out.println("Growth Rate: " + (growthRate * 100) + "%");

        System.out.println("Years: " + years);

        System.out.println("\n--- Recursive Approach ---");

        long startTime = System.nanoTime();

        double fvRecursive = futureValueRecursive(presentValue,
growthRate, years);

        long endTime = System.nanoTime();

        System.out.println("Future Value: $" + String.format("%.2f",
fvRecursive));

        System.out.println("Time taken: " + (endTime - startTime) + "
ns");

        System.out.println("\n--- Iterative Approach (Optimized) ---");

        startTime = System.nanoTime();

        double fvIterative = futureValueIterative(presentValue,
growthRate, years);

        endTime = System.nanoTime();

```

```

        System.out.println("Future Value: $" + String.format("%.2f",
fvIterative));

        System.out.println("Time taken: " + (endTime - startTime) + "
ns");

        System.out.println("\n--- Memoized Approach (Optimized) ---");
        java.util.Arrays.fill(memo, 0);
        startTime = System.nanoTime();

        double fvMemoized = futureValueMemoized(presentValue,
growthRate, years);

        endTime = System.nanoTime();

        System.out.println("Future Value: $" + String.format("%.2f",
fvMemoized));

        System.out.println("Time taken: " + (endTime - startTime) + "
ns");

        System.out.println("\n--- Complexity Analysis ---");

        System.out.println("Recursive:      O(n) time, O(n) stack
space");

        System.out.println("Iterative:      O(n) time, O(1) space");

        System.out.println("Memoized:      O(n) time, O(n) space (avoids
recomputation)");

        System.out.println("\nOptimization: Use iterative approach for
best performance,");

        System.out.println("or memoization when recursive logic is
complex.");
    }
}

```


Output:

```
> javac FinancialForecasting.java && java FinancialForecasting
=== Financial Forecasting Tool ===

--- Parameters ---
Present Value: $10000.0
Growth Rate: 8.0%
Years: 10

--- Recursive Approach ---
Future Value: $21589.25
Time taken: 2625 ns

--- Iterative Approach (Optimized) ---
Future Value: $21589.25
Time taken: 2625 ns

--- Memoized Approach (Optimized) ---
Future Value: $21589.25
Time taken: 2125 ns

--- Complexity Analysis ---
Recursive:    0(n) time, 0(n) stack space
Iterative:    0(n) time, 0(1) space
Memoized:     0(n) time, 0(n) space (avoids recomputation)

Optimization: Use iterative approach for best performance,
or memoization when recursive logic is complex.
```